

结构化编程 III — 方法

面向对象编程 · 2026 · 南京大学

| 刘钦

Outline

本讲是结构化编程三讲的第三讲（行为层面），前两讲分别讲了设计方法和数据。

- 方法的概念
 - 三视角 · 属性 · 行为
- 方法的实现
 - 声明 · 类图 · 调用机制 · Dog案例
- 参数与返回
 - Parameters/Arguments · 值传递 · Getter/Setter
- 方法体结构
 - 表达式 · 语句 · 控制流 · break标签
- 方法常见用法
 - 重载 · 常见方法模式
- 内存与虚拟机
 - Linux/JVM/C栈帧/Java方法执行
- 总结与展望

方法的概念

概念 · 属性 · 行为

方法的概念

- 方法的概念
 - 物理的角度：指令块
 - 逻辑的角度：抽象指令单元
 - 语义的角度：行为
- 分类
 - 类的行为：静态方法
 - 对象的行为：成员方法

方法对象的属性

- 名字
- 所有者
- 地址
- 接口
- 实现
- 运行期
- 可见性

属性 1~2 — 命名与所有者

属性 1 — 命名

- 第一个单词小写，其后每个单词首字母大写（小驼峰）
- 一般用动词或动宾短语： `getValue`， `setValue`， `isValid`

属性 2 — 所有者

- **类的行为**（静态方法）→ 通过类名调用
- **对象的行为**（成员方法）→ 通过对象引用调用

属性 3 — 接口

- 返回值
- 方法名
- 参数
 - 个数
 - 类型
 - 顺序
- 抛出的异常

```
// parseInt 接口示例
public static int parseInt(String s)
    throws NumberFormatException

// Parameters:
//   s - String containing int
// Returns:
//   the integer value
// Throws:
//   NumberFormatException
```

属性 4~6

- 属性 4 — 实现： `{}` 内的语句
- 属性 5 — 运行期
 - 同步：调用方法等待被调用方法执行完
 - 异步：调用方法和被调用方法不相关
- 属性 6 — 可见性

Access Levels

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
no modifier	Y	Y	N	N
private	Y	N	N	N

方法的实现

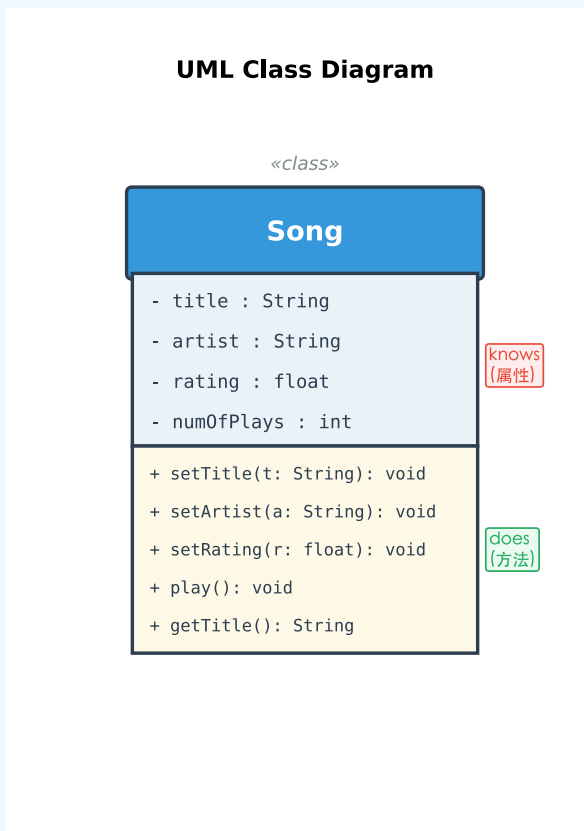
声明 · 类图 · 调用 · Dog案例

方法的声明与类图

方法的声明

```
修饰词  返回类型  方法名称(参数列表) {  
    方法体  
}
```

UML 类图



knows = 属性, does = 方法

成员方法的调用

```
void play() {  
    soundPlayer.playSound(title);  
}
```

方法被同步调用的特性：

- 每个方法都只有一个入口。
- 当执行被调用的方法的时候，调用方法暂停。
- 当方法结束时，程序的控制权交还给调用处。

Dog

```
public class Dog {  
    int size;  
    String name;  
    public void bark() {  
        if (size > 60) {  
            System.out.println("Woof! Woof!");  
        } else if (size > 14) {  
            System.out.println("Ruff! Ruff!");  
        } else {  
            System.out.println("Yip! Yip!");  
        }  
    }  
    public static void main(String[] args) {  
        Dog one = new Dog(); one.size = 70;  
        Dog two = new Dog(); two.size = 8;  
        Dog three = new Dog(); three.size = 35;  
        one.bark(); two.bark(); three.bark();  
    }  
}
```

参数与返回

Parameters · Arguments · 值传递 · Getter/Setter

Parameters and arguments

A method uses parameters. A caller passes arguments.

```
public class Dog2 {  
    public static void main(String[] args) {  
        Dog2 d = new Dog2();  
        d.bark(3);  
    }  
    void bark(int numofBarks) {  
        for (int i = 0; i < numofBarks; i++) {  
            System.out.println("Bark!");  
        }  
    }  
}
```

Return values

If you define a return value, you must return one.

```
public int getSize() {  
    return size;  
}  
public void setSize(int s) {  
    size = s;  
}  
public static void main(String[] args) {  
    Dog one = new Dog(); one.size = 70;  
    one.bark();  
    int size = one.getSize();  
    System.out.println("Size: " + size);  
}
```

Multi arguments

```
public void init(int size, String name) {  
    this.size = size;  
    this.name = name;  
}  
public static void main(String[] args) {  
    Dog one = new Dog(); one.size = 70;  
    one.bark();  
    int size = one.getSize();  
    System.out.println("Size: " + size);  
    one.init(50, "Rex"); // pass by value  
    System.out.println("Name: " + one.name);  
    System.out.println("Size: " + one.size);  
}
```

Java 只有值传递

基本类型：传递值的副本

```
void doubleIt(int x) {  
    x = x * 2; // 只改了副本  
}  
  
int a = 5;  
doubleIt(a);  
// a 仍然是 5
```

引用类型：传递引用的副本（指向同一个对象）

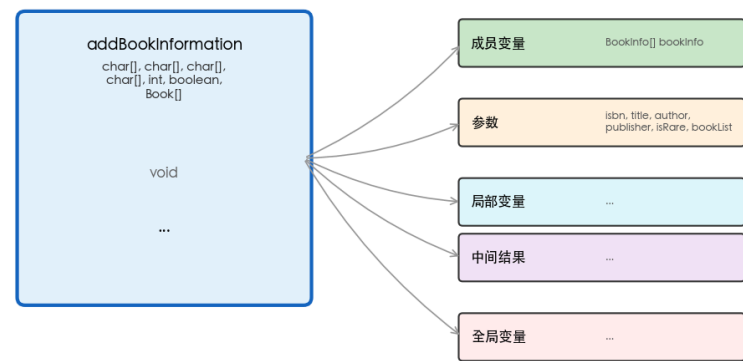
```
void rename(Dog d) {  
    d.setName("Max"); // 修改了对象 ✓  
}  
  
void reassign(Dog d) {  
    d = new Dog(); // 只改了副本 ✗  
}
```

Java 中没有"引用传递"——传递的是引用的值（地址的副本），不是对象本身。

Getter / Setter 与数据访问

```
public class Dog {  
    int size;  
    String name;  
    public int getSize() { return size; }  
    public void setSize(int s) { size = s; }  
    public String getName() { return name; }  
    public void setName(String n) { name = n; }  
}
```

方法访问数据的示例 — addBookInformation



方法体结构

表达式 · 语句 · 块 · 控制流

表达式、语句、块

- **Expression（表达式）** — 由变量、操作符、方法调用组成，求值产生一个值
- **Statement（语句）** — 完整的执行单元，以分号结尾
 - 表达式语句：赋值、++/--、方法调用、对象创建
 - 声明语句
 - 控制流语句
- **Block（块）** — 花括号 `{ }` 之间的零或多条语句，可以出现在任何单条语句允许的地方

三种语句

分支

```
// if-else
if (score >= 60) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}

// switch
switch (day) {
    case 1: name = "Mon"; break;
    case 2: name = "Tue"; break;
    default: name = "?";
}
```

循环

```
// for
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}

// while
while (hasNext()) {
    process(next());
}

// for-each
for (String s : list) {
    System.out.println(s);
}
```

break & continue

- **break**
 - breaks out of the inner iteration and you end up in the outer iteration.
 - break label (breaks all the way out to label, but it does not reenter the iteration)
- **continue**
 - the continue moves back to the beginning of the inner iteration.
 - continue label (reenter the iteration)

break 标签

```
int i = 1, j = 1, k = 1, n = 0;
boolean flag = false;
Loop1: for (i = 1; i <= 10; i++) {
    Loop2: for (j = 1; j <= 10; j++) {
        Loop3: for (k = 1; k <= 10; k++) {
            if (i + j + k == 10) {
                flag = true;
                break Loop1;
            }
            if (k >= 1) {
                Loop4: n = 1;
            }
        }
    }
}
/* 结果为: i=1, j=1, k=8, flag=true */
Loop5: for (int m = 1; m <= 10; m++) { ... }
```

break加标签只能用于跳出包含break语句的块，不能跳入其它块。break Loop4 或 break Loop5 都是非法的。

return 与 exit

return

- Specifies what value a method will **return**
- Causes the current method to **exit**

exit

- Exit code: 0 = fine; $\neq 0$ = error

```
public static void main(String... args) {  
    try {  
        runMyApp();  
    } catch (Exception e) {  
        System.exit(1);  
    }  
}
```

方法常见用法

重载 · 常见方法模式

方法重载 (Method Overloading)

重载：在一个类里面，不同的方法，恰好名字相同

```
void printInformation(int i) { ... }  
void printInformation(char[] cArray) { ... }  
    /* 与上面的方法参数类型不同 */  
void printInformation(char[] cArray, int i) { ... }  
    /* 与上面的方法参数个数不同 */  
void printInformation(int i, char[] cArray) { ... }  
    /* 与上面的方法参数顺序不同 */
```

非法的重载

- 参数标示符不同

```
void printInformation(int i) { ... }  
void printInformation(int j) { ... }    /* 非法! 仅参数名不同 */
```

- 在C++、Java、C#等语言中不同的返回类型不能用于区分方法:

```
int setBorrowedNum(int i) { ... }  
boolean setBorrowedNum(int j) { ... }    /* 非法! 仅返回类型不同 */
```

常见方法与常用方法

常见方法模式

- Boolean Setting — `valid()`, `invalid()`, `setValidity(boolean)`
- Query — `isVisible()`
- Equality — `equals()`, `hashCode()`

常用方法模式

- Getting — `getX()`
- Setting — `setX()`
- Convention

```
class Polar {  
    Cartesian asCartesian() { ... }  
}
```

递归与栈

递归 vs 迭代 · 栈溢出 · 尾递归 · SICP 视角

递归 — 方法调用自身

经典案例：阶乘

```
static int factorial(int n) {  
    if (n <= 1) return 1;    // 基础情况  
    return n * factorial(n - 1); // 递归调用  
}
```

调用展开

```
factorial(4)  
= 4 * factorial(3)  
= 4 * 3 * factorial(2)  
= 4 * 3 * 2 * factorial(1)  
= 4 * 3 * 2 * 1  
= 24
```

递归三要素

要素	说明	本例
基础情况	终止条件	n <= 1
递归调用	调用自身	factorial(n-1)
问题缩小	每次更接近终止	n 每次减 1

调用栈示意

```
| factorial(1)→1 | ← 栈顶  
| factorial(2)   | 等待 f(1)  
| factorial(3)   | 等待 f(2)  
| factorial(4)   | 等待 f(3)  
| main()         | ← 栈底
```

递归 vs 迭代 — SICP 的视角

递归过程（线性递归）

```
// 先展开，再收缩
static int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

f(4)→4*f(3)→3*f(2)→2*f(1)→1
 ←2←6←24

每层保留 `n *` 上下文 → $O(n)$ 栈空间。

迭代过程（尾递归/循环）

```
// 用累积器，无需展开
static int factIter(int n) {
    int product = 1;
    for (int i = 1; i <= n; i++)
        product *= i;
    return product;
}
```

每步状态由当前变量完整描述 → $O(1)$ 栈空间。

SICP: "不要混淆递归过程（计算形状）和递归语法（函数调用自身）。迭代过程可以用递归语法表达。"

StackOverflowError — 递归太深会怎样？

触发场景

```
static int badFactorial(int n) {  
    // 忘了基础情况!  
    return n * badFactorial(n - 1);  
}  
// badFactorial(5) → 无限递归 → 崩溃
```

报错信息

```
Exception in thread "main"  
java.lang.StackOverflowError  
    at Main.badFactorial(Main.java:3)  
    at Main.badFactorial(Main.java:3)  
    at Main.badFactorial(Main.java:3)  
    ... (重复数千行)
```

为什么会溢出？

每次方法调用都会在 JVM 栈上创建一个**栈帧**：

- 局部变量表
- 操作数栈
- 返回地址

栈空间有限（默认 512KB~1MB），递归太深 → 栈帧堆满 → **StackOverflowError**

与大作业的联系

```
eval( (+ 1 (+ 2 (+ 3 (+ 4 ...)))) )
```

大作业中 **eval** 递归调用自身——如果表达式嵌套太深，也会栈溢出。这就是 **Project 07 TCO**（尾调用优化）要解决的问题。

尾递归优化 (TCO) 预览

什么是尾调用?

```
// ❌ 非尾递归: return 后还要做 n *
static int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}

// ✅ 尾递归: return 直接返回递归结果
static int factTail(int n, int acc) {
    if (n <= 1) return acc;
    return factTail(n - 1, n * acc);
}
```

尾调用: 递归调用是方法的**最后一个操作**, 返回后无需再做任何事。

TCO 的效果

	普通递归	尾递归+TCO
栈空间	O(n)	O(1)
n=10000	StackOverflow	正常运行
原理	每层保留栈帧	复用同一栈帧

Java 不支持 TCO!

Java/JVM **不自动优化尾递归**——必须手动改写为循环。
但大作业的 Lisp 解释器中, **Project 07 TCO** 会用 `while(true)` 循环模拟尾调用优化:

```
// eval 中的 TCO 模式
while (true) {
    obj = handleDo(list, env); // 不 return
    // 继续循环, 复用当前栈帧
}
```

可变参数（varargs） — Java 5+

```
// Java 5 引入。... 表示接受 0 到 N 个参数，编译器自动打包成数组
static int sum(int... numbers) {
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}

sum();           // 0
sum(1, 2);       // 3
sum(1, 2, 3, 4); // 10
```

规则

规则	说明
只能有一个 varargs 参数	(int... a, int... b) ✗
必须是最后一个参数	(int... a, String s) ✗
本质是数组	int... ≡ int[]
main 就是 varargs	main(String... args) ✓

你已经在用了：public static void main(String[] args) 等价于 main(String... args)。

static 方法 vs 实例方法

static 方法（类方法）

```
class MathUtil {  
    static int add(int a, int b) {  
        return a + b;  
    }  
}
```

// 通过类名调用，无需创建对象

```
MathUtil.add(1, 2);
```

- 不依赖对象状态
- 不能访问 `this`
- 不能访问实例变量
- 典型用途：工具方法、`main`

实例方法

```
class Dog {  
    int size;  
    void bark() {  
        // 可以访问 this.size  
        if (size > 60) { ... }  
    }  
}
```

// 必须先创建对象

```
Dog d = new Dog();  
d.bark();
```

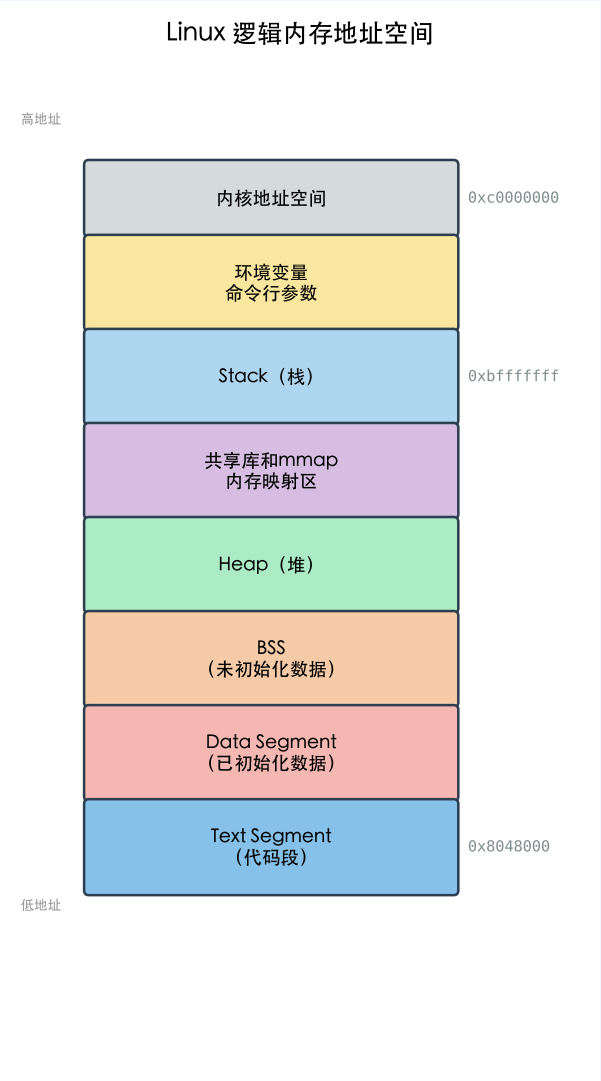
- 依赖对象状态（`this`）
- 可以访问实例变量
- 典型用途：操作对象数据

选择依据：方法需要访问对象状态 → 实例方法；不需要 → `static` 方法。大作业中 `Evaluator.eval()` 是 `static`，因为它不依赖 `Evaluator` 对象的实例状态。

内存与虚拟机

Linux内存 · JVM内存 · C栈帧 · Java方法执行

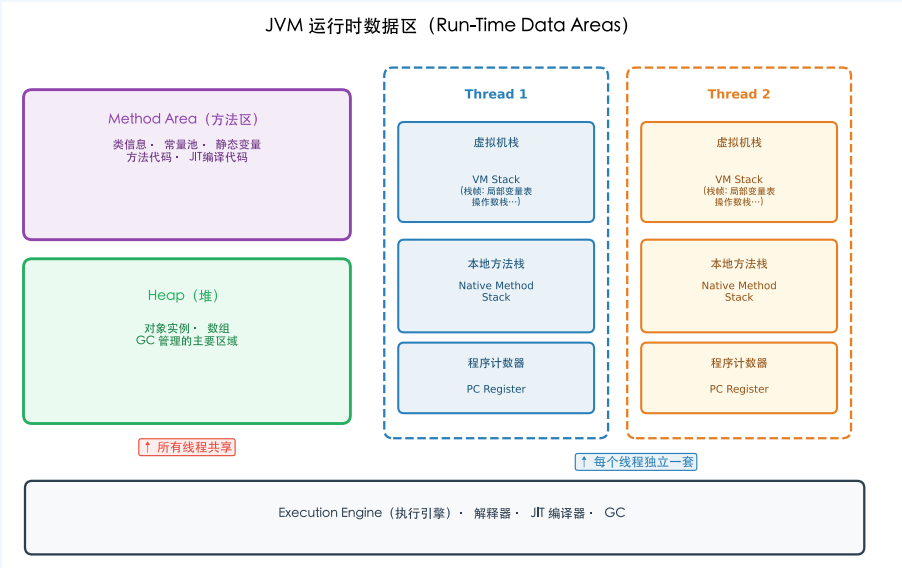
Linux 逻辑内存地址空间



Program Memory 各段

段	内容
Code	程序指令
Data	已初始化的全局/静态变量
BSS	未初始化数据
Heap	动态分配 <code>malloc</code> / <code>new</code>
Stack	方法调用的局部变量

JVM 运行时数据区



各区域说明

区域	线程共享	内容
Method Area	共享	类信息、常量池、静态变量
Heap	共享	对象实例、数组
VM Stack	独立	栈帧（局部变量、操作数栈）
Native Stack	独立	本地方法调用
PC Register	独立	当前执行的字节码地址

引用类型变量对Java对象访问的实现

- 访问数据
 - 实例数据值（对象中各个实例字段的数据）
 - 对象类型数据（对象类型、父类、实现的接口、方法等）
- 访问方式
 - **句柄访问方式** — Java堆中划出句柄池，reference → 句柄 → 实例数据 + 类型数据
 - **直接指针访问方式** — reference 直接指向对象实例数据（含类型数据指针）

C语言方法调用的栈帧

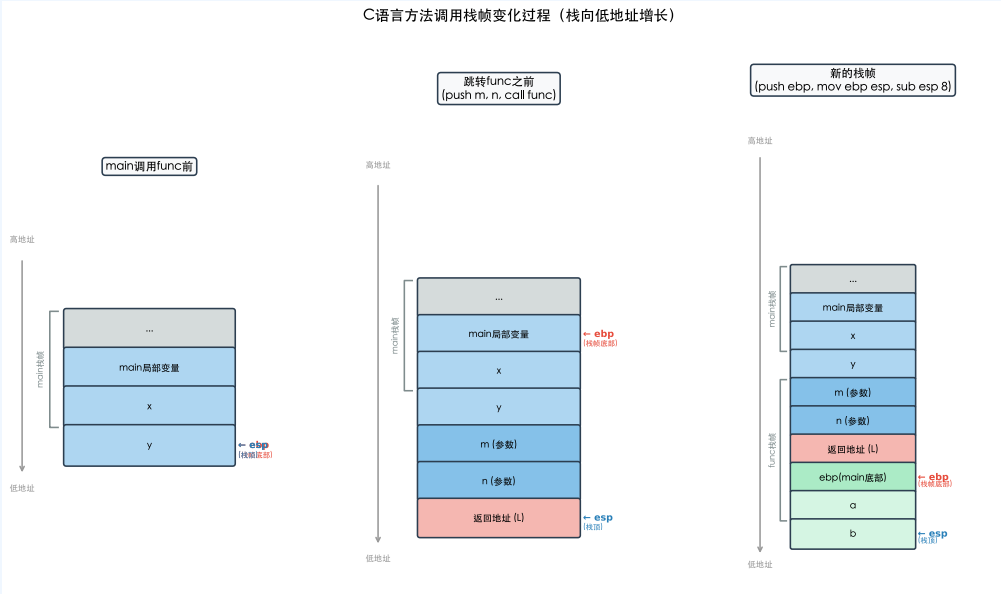
push · call · ebp/esp · ret

C语言方法调用 — 示例代码与栈帧变化

示例代码

```
void func(int m, int n) {  
    int a, b;  
    a = m;  
    b = n;  
}  
  
main() {  
    ...  
    func(x, y);  
    L: // 下一条语句  
    ...  
}
```

栈帧变化过程



C 栈帧 — 调用与建立

1. main 调用 func 时

- `push m` / `push n` — 两个参数压入栈
- `call func` — 将返回地址压栈，跳转到 func

2. func 开始

- `push ebp` — 保存调用者的栈帧底部（为了返回时恢复）
- `mov ebp, esp` — 上一栈帧的顶部 = 新栈帧的底部
- `sub esp, 8` — 为局部变量 `int a, b` 分配 8 字节空间

C 栈帧 — 赋值与返回

3. 赋值

- `mov [esp+4], [ebp+12] → a = m` (通过 `ebp` 定位参数)
- `mov [esp], [ebp+8] → b = n`

4. 返回

- `ret 8` — 弹出返回地址, `esp` 跳过参数空间 (8 字节), 释放 `m, n`
- `ebp` 恢复为调用者的值 → 栈帧完全恢复到调用前的状态

核心机制: `ebp` 定位参数和局部变量, `esp` 跟踪栈顶。调用时 `push` 建立新帧, 返回时 `pop` 恢复旧帧。

Java方法的执行

栈架构

广义解析器

源代码: `i = a + b * c`

↓ (词法分析)

单词流: `[i] [=] [a] [+] [b] [*] [c]`

↓ (语法分析)

语法树(AST):

```
      =  
     /\br/>    i  +  
     /\br/>    a  *  
     /\br/>    b  c
```

示例代码 1

```
public class EvalOrderDemo {  
    public static void main(String[] args) {  
        int[] arr = new int[1];  
        int a = 1;  
        int b = 2;  
        arr[0] = a + b;  
    }  
}
```

Bytecode — 示例 1

```
// 左子树: 数组下标  
// a[0]  
aload_1  
iconst_0
```

```
// 右子树: 加法  
// a  
iload_2
```

```
// b  
iload_3
```

```
// +  
iadd
```

```
// 根节点: 赋值  
iastore
```

从根节点开始**后序遍历**准备生成代码

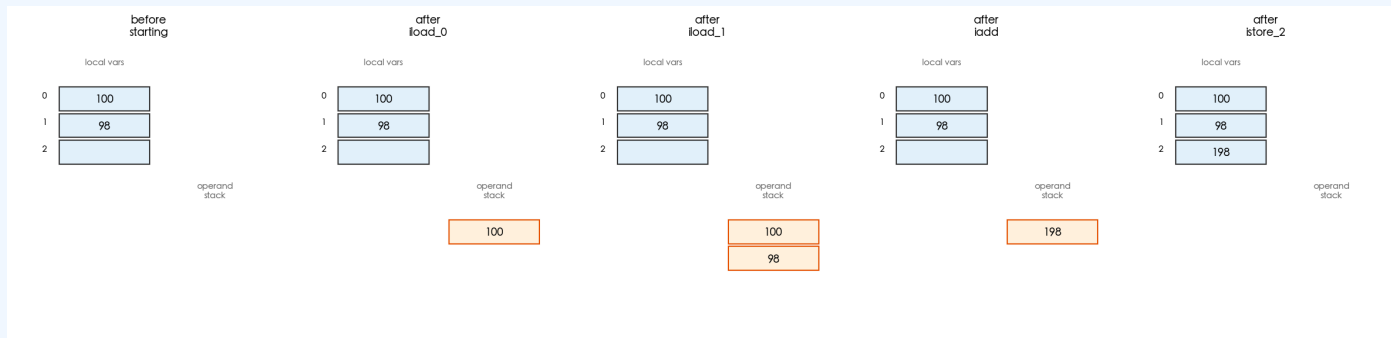
示例代码 2

```
public class Demo {  
    public static void foo() {  
        int a = 1;  
        int b = 2;  
        int c = (a + b) * 5;  
    }  
}
```

Bytecode — 示例 2

- 0: iconst_1
- 1: istore_0
- 2: iconst_2
- 3: istore_1
- 4: iload_0
- 5: iload_1
- 6: iadd
- 7: iconst_5
- 8: imul
- 9: istore_2
- 10: return

Operand Stack



```
iload_0 // push the int in local variable 0
iload_1 // push the int in local variable 1
iadd    // pop two ints, add them, push result
istore_2 // pop int, store into local variable 2
```

JVM 运行时栈帧结构

每个线程有自己的栈，栈帧包含：

- 局部变量表 (Local Variable Table)
- 操作数栈 (Operand Stack)
- 动态连接 (Dynamic Linking)
- 返回地址 (Return Address)

栈帧随方法调用创建，随方法返回销毁。

Method parameters on local variables

runClassMethod (static)

index	type	parameter
0	int	int i
1	long	long l
3	float	float f
4	double	double d
6	reference	Object o
7	int	byte b

runInstanceMethod (instance)

index	type	parameter
0	reference	hidden this
1	int	char c
2	double	double d
4	int	short s
5	int	boolean b

实例方法的 index 0 总是 hidden this 引用

Sample Code — Allocating Frames

```
class Example3c {  
    public static void addAndPrint() {  
        double result = addTwoTypes(1, 88.88);  
        System.out.println(result);  
    }  
    public static double addTwoTypes(int i, double d) {  
        return i + d;  
    }  
}
```

调用过程:

1. **before invocation** — addAndPrint 栈帧中操作数栈压入 1 和 88.88
2. **after invocation** — 创建 addTwoTypes 栈帧, 局部变量区存 i=1, d=88.88
3. **after returns** — addTwoTypes 栈帧销毁, 结果 89.88 返回到 addAndPrint 操作数栈

结构化编程总结与展望

结构化编程三讲回顾

结构化编程范式

- |
 - 第06讲 · 设计层面
 - 自顶向下分解 → 数据流图 → 结构图 → 流程图
 - 第07讲 · 数据层面
 - 变量 → 赋值 → 类型系统 → 生存期/作用域
 - 第08讲 · 行为层面
 - 方法 → 参数/返回 → 控制流 → 重载 → 内存模型

核心公式：结构化编程 = 自顶向下分解 + 三种控制结构 + 全局数据

结构化编程的优缺点

优点	缺点
自顶向下分解，逻辑清晰	数据与行为分离 ：全局数据谁都能改
三种控制结构表达任意算法	修改波及面大 ：改一个数据结构，所有相关方法都要改
模块化设计，适合流程导向问题	难以建模现实世界 ：现实世界是"对象"而非"流程"
入门门槛低，容易理解	规模受限 ：系统变大后全局数据和方法数量爆炸

为什么需要面向对象？

结构化风格

```
// 数据和行为分离
String[] bookTitles = new String[100];
boolean[] bookBorrowed = new boolean[100];

void borrowBook(int i) { ... }
void returnBook(int i) { ... }
```

- 数据暴露，任何方法都能修改
- 加字段 → 所有方法都要改

面向对象风格

```
// 数据和行为封装在一起
class Book {
    private String title;
    private boolean borrowed;
    public void borrow() { ... }
    public void returnBook() { ... }
}
```

- 数据被保护，通过接口访问
- 改内部 → 接口不变则外部无影响

下一讲起，我们正式进入面向对象编程的世界。